



sensors-factory

Control for configuring multiple sensors with associated colors, typically for charts and graphs. Requires the [Sensors plugin](#).

Attributes

Attribute	Type	Description	Support JS expressions
<code>data-default</code>	<code>string</code>	Default sensor ID for new entries	

Output Value

`array` of objects:

Property	Type	Description
<code>sensorId</code>	<code>string</code>	Sensor identifier
<code>color</code>	<code>string</code>	Associated color (hex)

Requirements

This control requires the Sensors plugin to be declared in manifest.json:

```
"required_plugins": [  
  "widgetbuilder.sensorsdataprowider:Sensors:1.0"  
]
```

Example

Manifest:

```
"required_plugins": [  
  "widgetbuilder.sensorsdatapvider:Sensors:1.0"  
]
```

HTML:

```
<meta  
  name="x-icue-property"  
  content="sensors"  
  data-label="tr('Sensors')"  
  data-type="sensors-factory"  
  data-default="plugins.Sensorsdatapvider.getDefaultSensorIdBlock('temperature')"  
>
```

Usage in JavaScript

```
console.log(sensors);  
// [{sensorId: "cpu_temp_0", color: "#FF0000"}, {sensorId: "gpu_temp_0", color:  
"#00FF00"}]  
  
sensors.forEach((item) => {  
  console.log(item.sensorId); // "cpu_temp_0"  
  console.log(item.color); // "#FF0000"  
});
```

Complete Widget Example

Manifest:

```
{  
  "author": "Corsair Team",  
  "id": "com.corsair.sensorsfactorydemo",  
  "name": "Sensors Factory Demo",
```

```

"description": "Sensors Factory Demo",
"version": "1.0.0",
"preview_icon": "icon.png",
"min_framework_version": "1.0.0",
"os": [
  {
    "platform": "windows"
  },
  {
    "platform": "mac"
  }
],
"supported_devices": [
  {
    "type": "dashboard_lcd"
  },
  {
    "type": "pump_lcd"
  },
  {
    "type": "keyboard_lcd"
  }
],
"interactive": true,
"required_plugins": [
  "widgetbuilder.sensorsdataprovder:Sensors:1.0"
]
}

```

HTML:

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>'Sensors Factory Demo'</title>
    <link rel="icon" type="image/svg+xml" href="resources/icon.svg" />

    <meta
      name="x-icue-property"
      content="sensors"
      data-label="'Sensors'"
      data-type="sensors-factory"
      data-
default="plugins.Sensorsdataprovder.getDefaultSensorIdBlock('temperature')"
    />

```

```

<script type="application/json" id="x-icue-groups">
  [{ "title": "'Settings'", "properties": ["sensors"] }]
</script>

<script src="common/plugins/IcueWidgetApiWrapper.js"></script>
<script src="common/plugins/SimpleSensorApiWrapper.js"></script>
</head>
<body
  style="margin:0;min-height:100vh;display:flex;flex-
wrap:wrap;gap:10px;padding:20px;box-sizing:border-
box;background:#1a1a2e;color:#fff;font-family:sans-serif;"
>
  <div id="list"></div>
  <script>
    let api = null;
    icueEvents = { onDataUpdated: update, onICUEInitialized: update };
    pluginSensorsdataprovderEvents = { onInitialized: onPluginReady };

    function onPluginReady() {
      api = new SimpleSensorApiWrapper(window.plugins.Sensorsdataprovder);

window.plugins.Sensorsdataprovder.sensorValueChanged.connect(updateValues);
      update();
    }

    async function updateValues() {
      if (!api || !sensors) return;
      for (const s of sensors) {
        const el = document.getElementById("v-" + s.sensorId);
        if (el) el.textContent = parseFloat(await
api.getSensorValue(s.sensorId)).toFixed(1);
      }
    }

    async function update() {
      if (!api || !sensors) return;
      const list = document.getElementById("list");
      list.innerHTML = "";
      for (const s of sensors) {
        const val = await api.getSensorValue(s.sensorId);
        const units = await api.getSensorUnits(s.sensorId);
        list.innerHTML +=
          '<div style="margin:5px;"><span id="v-' +
s.sensorId +
'" style="color:' +
s.color +
';font-size:24px;">' +
parseFloat(val).toFixed(1) +
"</span> " +

```

```
        units +  
        "</div>";  
    }  
}  
  
if (iCUE_initialized) update();  
if (typeof pluginSensorsdataprovder_initialized !== "undefined" &&  
pluginSensorsdataprovder_initialized)  
    onPluginReady();  
</script>  
</body>  
</html>
```

For the example to work, you also need to add icon.svg

Copy `common/plugins/` from the documentation bundle into your widget folder, then include wrapper files with `<script src=` as shown above.